

《OpenCV + Python: 图像与视频处理基础》

授课教师：王洪松

邮箱：hongsongsui@gmail.com

图像修补、模板匹配 Seam Carving、图像风格变换

- 图像修补涉及对图像中的损坏部分（如划痕、污渍、缺失区域等）进行修复。用修补技术去除照片中小的噪音或划痕。
- 图像修补应用：
 - 照片修复：修复老照片、家庭照片中的划痕、褪色等问题。
 - 艺术品修复：修复艺术品、油画等，使画面恢复原貌。
 - 医学影像处理：处理影像中的噪声、伪影等问题，提高影像质量。
- 图像修补方法：
 - 基于插值的方法：适用于图像的简单补全和缩放操作。这种方法通过计算邻近像素的值来估计缺失像素的值。
 - 基于全变分或偏微分方程的方法：通过最小化图像的全变分来进行修复。这种方法能够较好地保持图像的边缘信息，但在处理大面积缺失时可能效果不佳。
 - 基于深度学习的方法。

图像修补

<https://hongsong-wang.github.io/>

`cv2.inpaint(src, mask, radius, flags)`

mask: 掩码图像, 非零像素表示需要修复的区域。

radius: 算法考虑的每个修复点的圆形邻域的半径。

flags: opencv支持的修补方法, 有两个:

`cv2.INPAINT_TELEA`, Telea算法, 基于插值的方法。

`cv2.INPAINT_NS`, Navier-Stokes算法, 基于偏微分方程 (PDE) 的方法。

掩码图像可以人工绘制或者基于修补区域的特点编程生成。

```
img = cv2.imread('messi_2.jpg' )  
mask = cv2.imread('mask2.png',0)  
dst = cv2.inpaint(img,mask,3,  
cv2.INPAINT_TELEA)
```



- `cv2.calcHist()` 是 OpenCV 中一个用于计算图像直方图的函数。

`hist = cv2.calcHist(images, channels, mask, hist_size, ranges, accumulate)`

`images`: 源图像列表，一个包含图像的数组。这个列表通常只包含一个图像。

`channels`: 计算其直方图的通道索引的列表。例如，如果输入图像是灰度图，这个列表将是 `[0]`。对于彩色图像，你可以指定要计算直方图的通道，如 `[0]`、`[1]` 或 `[2]` 分别对应 BGR 通道中的蓝色、绿色和红色通道，或者 `[0, 1, 2]` 来计算所有通道的直方图。

`mask`: 这是一个可选参数，指定了一个与源图像相同大小和类型的掩码图像。要计算直方图的图像区域由掩码中非零像素指定。如果掩码为 `None`，则计算整个图像的直方图。

`hist_size`: 这是一个包含每个维度直方图大小的数组。对于灰度图像，这通常是一个包含单个元素的列表，例如，`[256]`。

`ranges`: 这是一个包含每个维度直方图边界的数组。对于灰度图像，这通常是 `[0, 256]`。

`accumulate`: 这是一个布尔值。如果设置为 `True`，则直方图将被累积，即多个图像的直方图将合并到一个直方图中。

2D 直方图

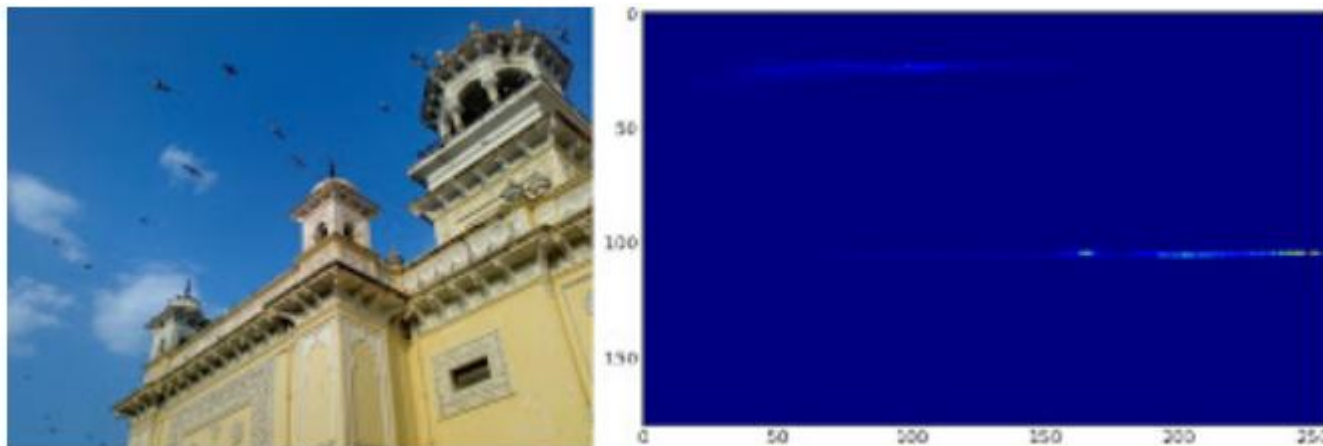
<https://hongsong-wang.github.io/>

- 使用函数 `cv2.calcHist()` 来计算直方图，首先需要将图像的颜色空间从 BGR 转换到 HSV。计算2D直方图，参数修改如下：
 - `channels=[0, 1]` 因为我们需要同时处理 H 和 S 两个通道。
 - `bins=[180, 256]` H 通道为 180, S 通道为 256。
 - `range=[0, 180, 0, 256]` H 的取值范围在 0 到 180, S 的取值范围在 0 到 256。

```
hsv = cv2.cvtColor(img,cv2.COLOR_BGR2HSV)
```

```
hist = cv2.calcHist([hsv], [0, 1], None, [180, 256], [0, 180, 0, 256])
```

输入图像和颜色直方图。
X 轴显示 S 值，Y 轴显示 H 值。



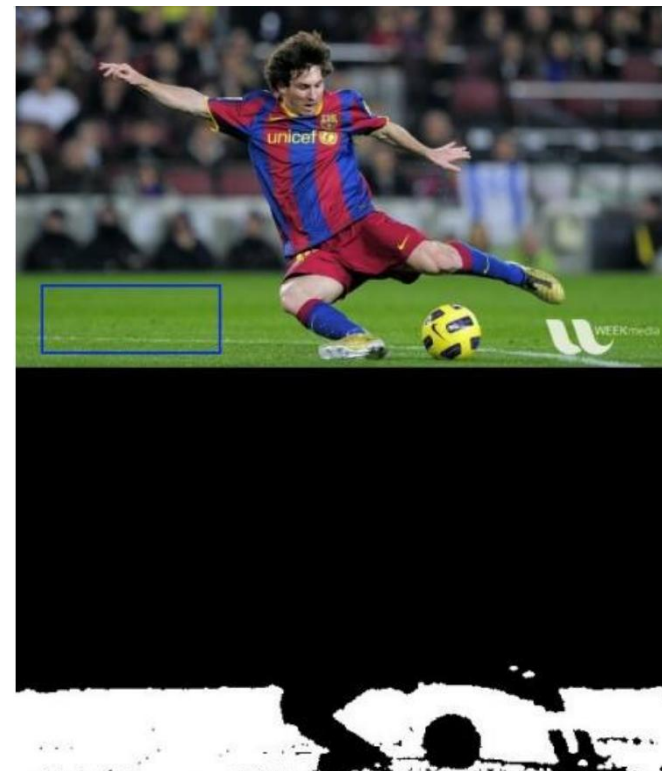
- 直方图反向投影是通过将颜色直方图反向投影到图像上，以突出显示图像中与该特征相似的区域。
 - 选择或定义要检测的目标对象，并计算其颜色直方图。
 - 对直方图进行归一化处理，使其成为一个概率分布。
 - 对于输入图像中的每个像素点，根据其颜色值在直方图中查找对应的概率值，并将该概率值作为反向投影的结果。

`cv2.calcBackProject()` 可以用来做直方图反向投影。它的参数与函数 `cv2.calcHist` 的参数基本相同。其中的一个参数是要查找目标的直方图。同样再使用目标的直方图做反向投影之前我们应该先 对其做归一化处理。返回的结果是一个概率图像。

直方图反向投影

<https://hongsong-wang.github.io/>

```
roi = cv2.imread('tar.jpg')
hsv = cv2.cvtColor(roi,cv2.COLOR_BGR2HSV)
target = cv2.imread('roi.jpg')
hsvt = cv2.cvtColor(target,cv2.COLOR_BGR2HSV)
roihist = cv2.calcHist([hsv],[0, 1], None, [180, 256], [0, 180, 0, 256] )
cv2.normalize(roihist,roihist,0,255,cv2.NORM_MINMAX) # normalize histogram
dst = cv2.calcBackProject([hsvt],[0,1],roihist,[0,180,0,256],1)
disc = cv2.getStructuringElement(cv2.MORPH_ELLIPSE,(5,5))
dst=cv2.filter2D(dst,-1,disc)
# threshold and binary AND
ret,thresh = cv2.threshold(dst,50,255,0)
thresh = cv2.merge((thresh,thresh,thresh))
res = cv2.bitwise_and(target,thresh)
res = np.hstack((target,thresh,res))
cv2.imshow('1',res)
cv2.waitKey(0)
```



模板匹配

<https://hongsong-wang.github.io/>

- 模板匹配是用来在一副大图中搜寻查找模版图像位置。

`cv2.matchTemplate()`, 和 2D 卷积一样, 模板图像在输入图像 (大图) 上滑动, 并在每一个位置对模板图像和与其对应的输入图像的子区域进行比较。

如果输入图像的大小是 $(W \times H)$, 模板的大小是 $(w \times h)$, 输出的结果的大小就是 $(W - w + 1, H - h + 1)$ 。当你得到这幅图之后, 就可以使用函数 `cv2.minMaxLoc()` 来找到其中的最小值和最大值的位置了。

```
res = cv2.matchTemplate(img, template, method)
```

```
min_val, max_val, min_loc, max_loc =  
cv2.minMaxLoc(res)
```

Matching Result



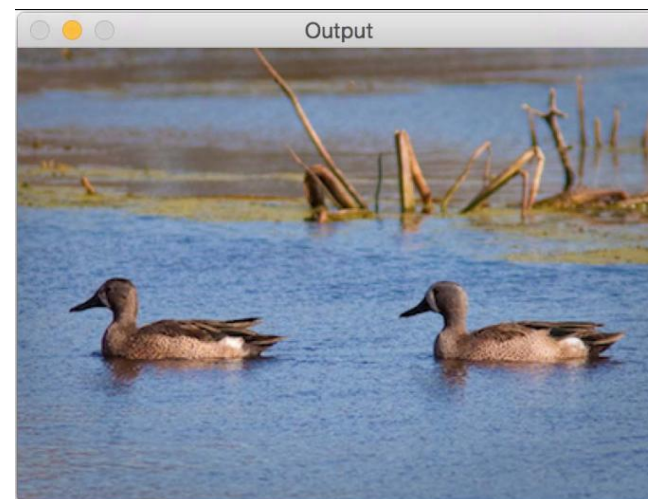
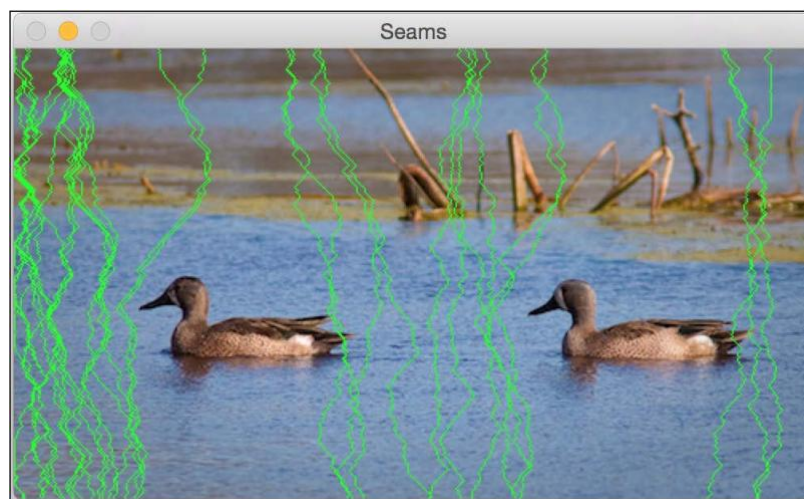
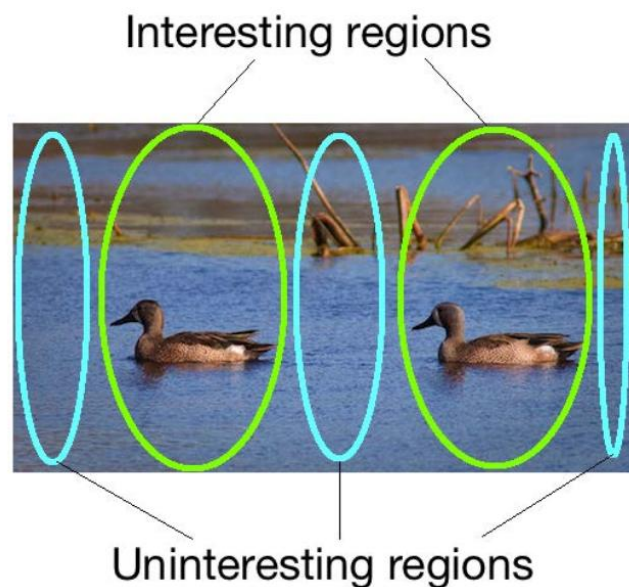
Detected Point



Seam Carving

<https://hongsong-wang.github.io/>

- 很自然的扩张图像，并且还保持了感兴趣的目标主体的形状比例。
- 使某个对象从图像中消失。



Seam Carving算法是通过移除或者插入累计能量最小的像素线的方式来实现图像缩放。每次在垂直或水平方向上利用动态规划法找到一条能量最小的像素线，通过删除该像素线实现图像缩小，反之插入该像素线实现图像放大。

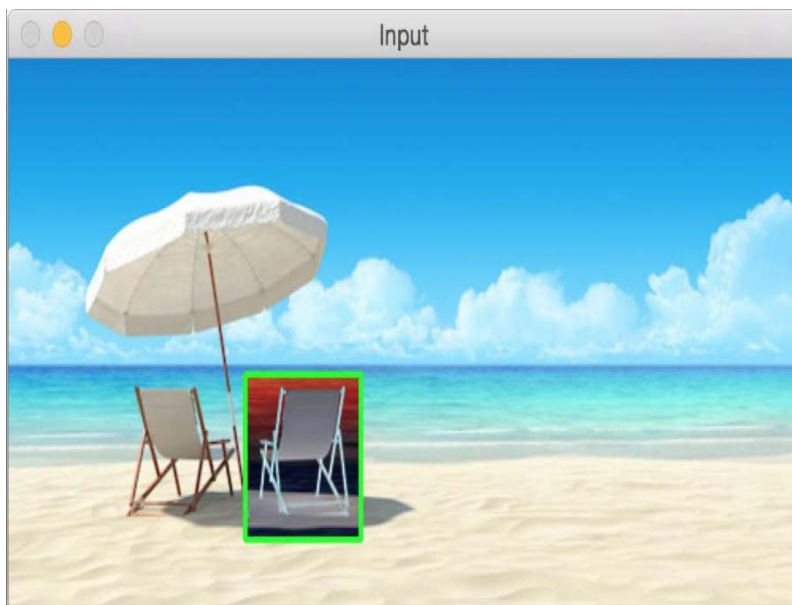
Seam Carving

<https://hongsong-wang.github.io/>

- 删除图像中的某个对象



原图



选择删除的区域



删除目标后的图像

使用Seam Carving去除物体。首先选择感兴趣的区域，让所有的接缝都穿过这个区域。在我们移除与该区域相关的所有接缝之后，继续添加接缝，直到将图像扩展至其原始宽度。

- 颜色查找表就是一种像素值映射的查找表。
- 调用LUT 函数可以获得最快的速度，这是因为OpenCV库可以通过英特尔线程架构启用多线程。

`cv2.LUT(src, lut, dst=None)`

src: 原始图像。

lut: 256维的向量，[0, 255]对应颜色值的查找表。如果输入src是多通道的，而查表是单通道的，则多通道使用的是同一个查找表。

例如，增加图像亮度用颜色查找表实现：

```
table = np.array([i * brightness_factor for i in range  
(0,256)]).clip(0,255).astype('uint8' )  
dst = cv2.LUT(img, table)
```

颜色查找表应用：风格变换

<https://hongsong-wang.github.io/>

```
from scipy.interpolate import UnivariateSpline

def LookupTable(x, y):
    spline = UnivariateSpline(x, y)
    return spline(range(256))

#winter effect
def Winter(img):
    increaseLookupTable = LookupTable([0, 64, 128, 256],
[0, 80, 160, 256])
    decreaseLookupTable = LookupTable([0, 64, 128, 256],
[0, 50, 100, 256])
    blue_channel, green_channel, red_channel = cv2.split(img)
    red_channel = cv2.LUT(red_channel,
decreaseLookupTable).astype(np.uint8)
    blue_channel = cv2.LUT(blue_channel,
increaseLookupTable).astype(np.uint8)
    win= cv2.merge((blue_channel, green_channel, red_channel))
    return win
```



铅笔素描

<https://hongsong-wang.github.io/>

- 生成类似铅笔的非真实的线条绘制的图像。

```
dst1_gray, dst1_color = cv2.pencilSketch(img, sigma_s = 50,  
sigma_r = 0.15, shade_factor = 0.04)
```

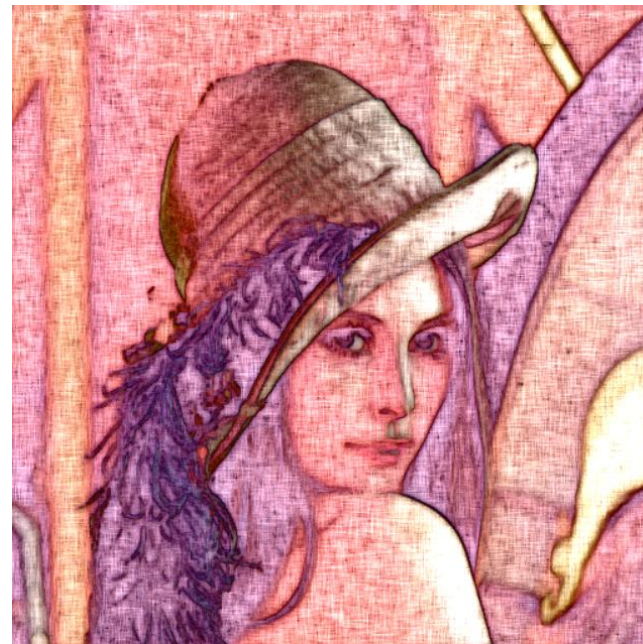
`sigma_s`: 邻近Pixel的数量,
范围在0到200之间。

`sigma_r`: 颜色平衡度, 数值越大, 同颜色的区域就会越大。范围在0到1之间。

`shade_factor`: 图像亮度, 值越高图像越亮, 范围在0到0.1之间。



dst1_gray



dst1_color

```
img = cv2.imread('LenaRGB.bmp', -1)
```

```
dst1_gray, dst1_color = cv2.pencilSketch(img, sigma_s  
= 50, sigma_r = 0.15, shade_factor = 0.04)
```

图像风格化

<https://hongsong-wang.github.io/>

- 制作具有各种各样效果的数字图像，而不是关注照片真实性。
- 抽出对比度低的特征，保留对比度高的特征。

```
dst = cv2.stylization(img, sigma_s=200, sigma_r=0.6)
```

`sigma_s`: 邻近Pixel的数量，范围在0到200之间。

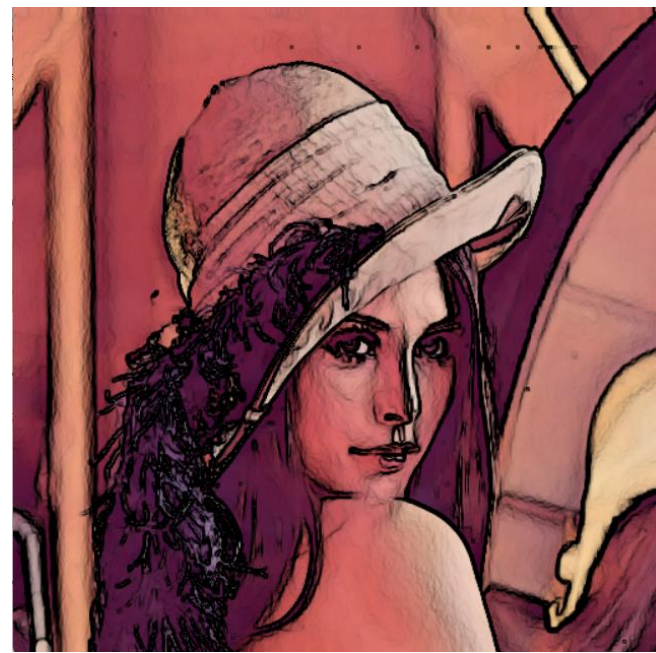
`sigma_r`: 颜色平衡度，数值越大，同颜色的区域就会越大。范围在0到1之间。

```
img = cv2.imread('LenaRGB.bmp', -1)
```

```
dst = cv2.stylization(img, sigma_s = 50,  
sigma_r = 0.15)
```



原图



风格化图像

图像细节增强

<https://hongsong-wang.github.io/>

- 对图像滤波以增强图像的细节。

```
dst = cv2.detailEnhance(img, sigma_s=100, sigma_r=0.1)
```

`sigma_s`: 邻近Pixel的数量, 范围在0到200之间。

`sigma_r`: 颜色平衡度, 数值越大, 同颜色的区域就会越大。范围在0到1之间。



原图



细节增强图像

```
img = cv2.imread('LenaRGB.bmp', -1)
dst = cv2.stylization(img, sigma_s = 50,
sigma_r = 0.15)
```

边缘保持图像平滑

<https://hongsong-wang.github.io/>

- 保留图片边缘特征做的平滑化处理，可以实现磨皮、美颜的效果。

```
dst = cv2.edgePreservingFilter(img, flags=1, sigma_s = 50, sigma_r = 0.15)
```

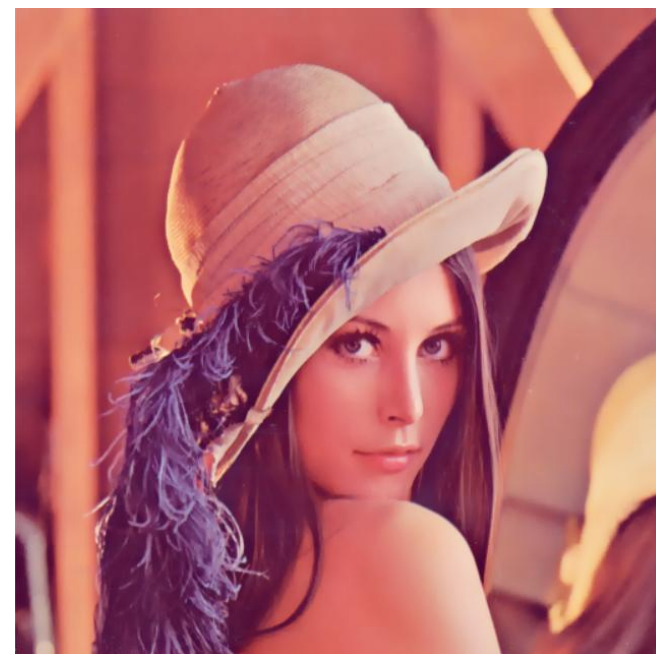
flags: 滤波参数, 1为递归双边滤波（速度3倍快），2为归一化卷积(对边缘产生锐化)。

sigma_s: 邻近Pixel的数量，范围在0到200之间。

sigma_r: 颜色平衡度，数值越大，同颜色的区域就会越大。范围在0到1之间。



原图



边缘保持图像平滑

```
img = cv2.imread('LenaRGB.bmp', -1)
dst = cv2.edgePreservingFilter(img,
flags=1, sigma_s = 50, sigma_r = 0.15)
```

- 去除图像中的文字
 - 在输入图像的背景区域添加文字。
 - 去除图像中文字，恢复文字区域的背景。
 - 分析什么场合去除文字的结果好，什么场合的结果不好。
- 放大/缩小图像中的物体
 - 输入图像，分割前景目标
 - 用滚动条控制该目标的大小，推动滚动条，该目标相应的放大或缩小
 - 目标缩小时，空出的白色背景需要用有效方法处理

练习题

<https://hongsong-wang.github.io/>

- 沿着曲线插入文字
 - 沿着右图彩虹的曲线边缘插入用中文写的一句话。
- 图像目标数数
 - 写程序自动算出下图中鸟的个数。



<https://img2.baidu.com/it/u=3540184563,2794363825&fm=253&fmt=auto&app=138&f=JPEG?w=750&h=500>